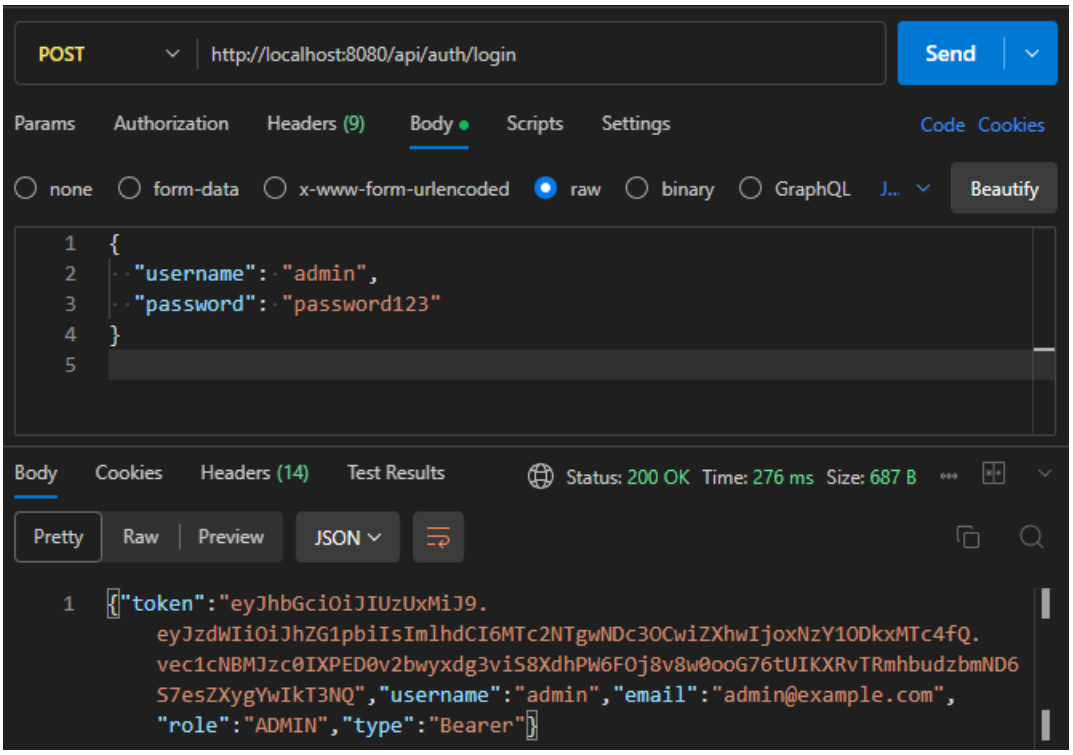
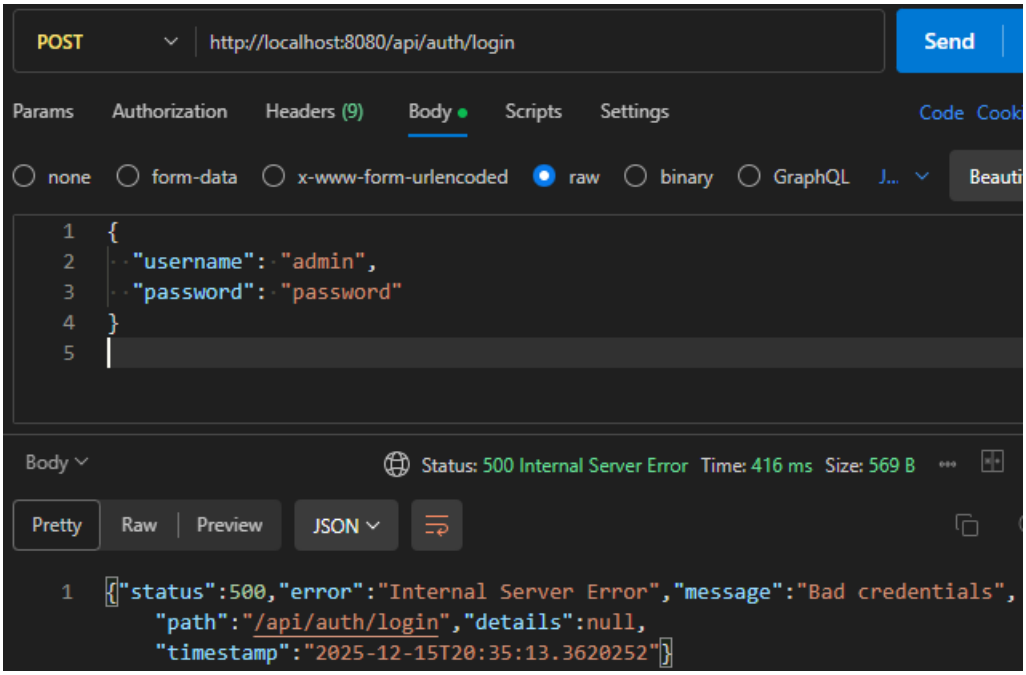
Part A:

Test 1:



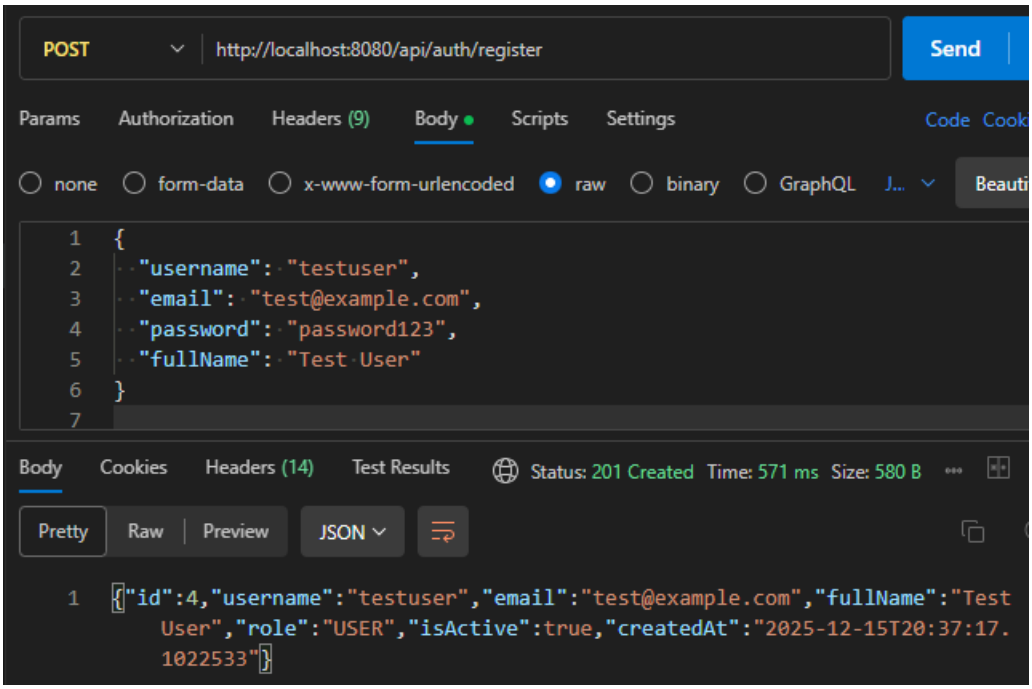
When valid credentials are provided, Spring Security authenticates the user using DaoAuthenticationProvider and the CustomUserDetailsService.
If authentication succeeds, the application generates a JWT token containing the username and expiration time and returns it to the client together with basic user information.

Test 2:



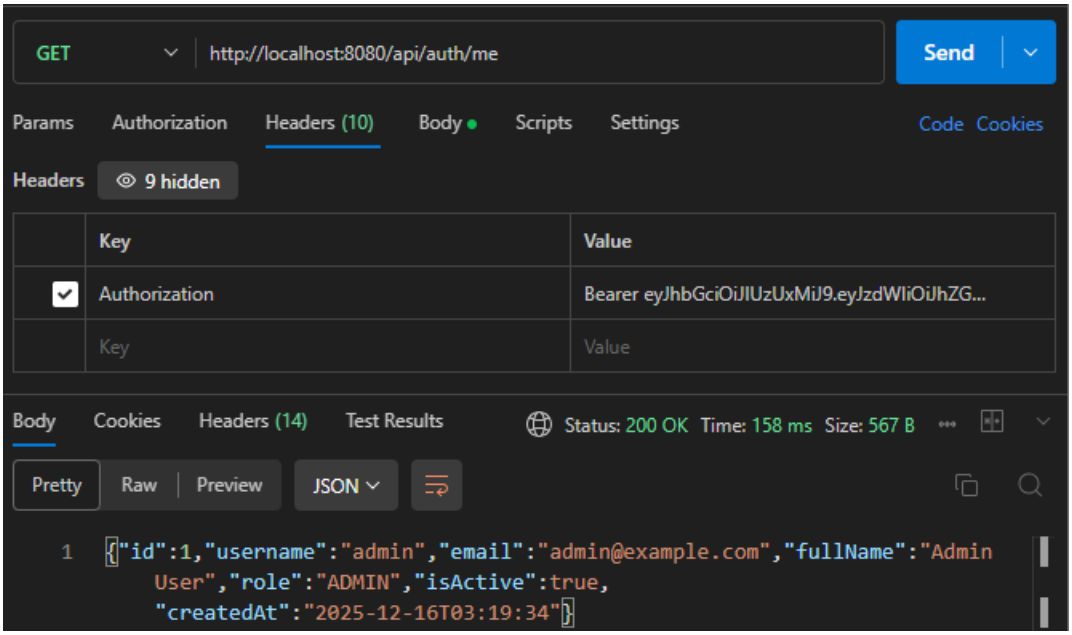
When the password does not match the BCrypt hash stored in the database, Spring Security throws a BadCredentialsException.
The application returns an error response (HTTP 401 Unauthorized or a similar error message, depending on the global exception handler).
This verifies that invalid credentials are correctly rejected and that the system does not allow unauthorized logins.

Test 3:



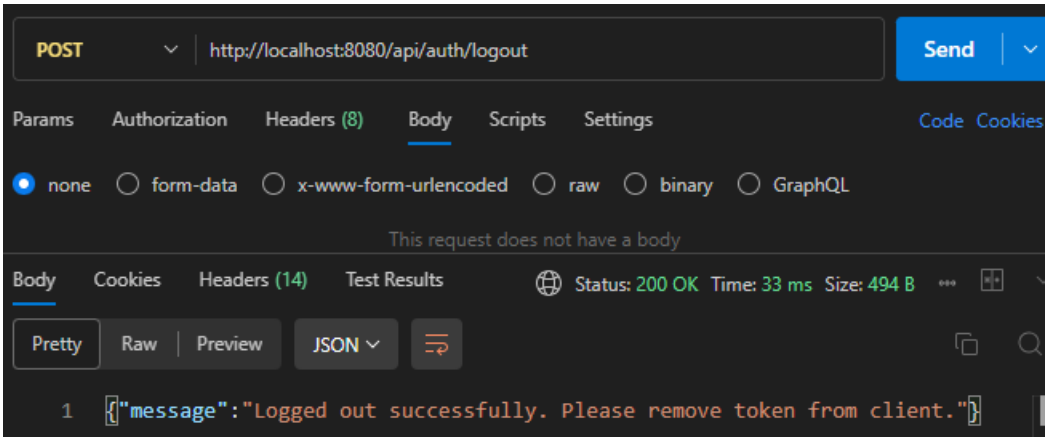
The register endpoint validates the input, checks for duplicate usernames and emails, and then encodes the password using BCrypt before saving the new user.
The response includes a UserResponseDTO with the created user's information and default role USER.
This confirms that new users can be safely registered with secure password storage.

Test 4:



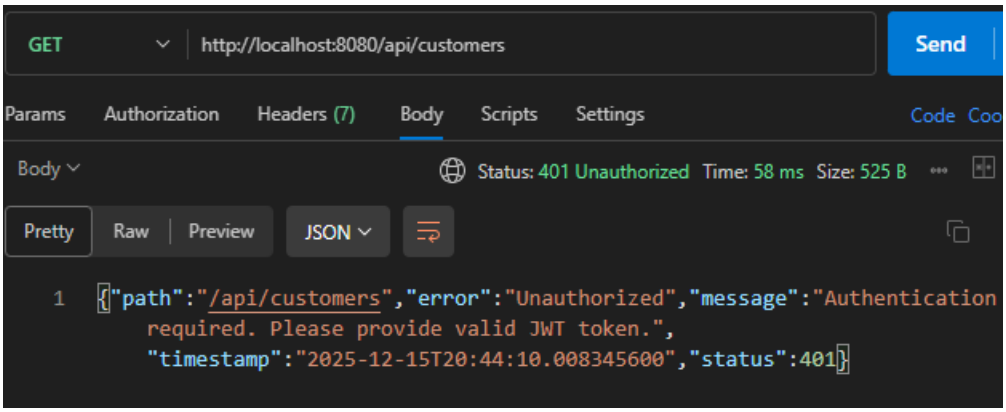
The /api/auth/me endpoint reads the authenticated user from the SecurityContextHolder and returns their information as a UserResponseDTO.
This demonstrates that the server can identify the currently logged-in user based on the JWT token and that the authentication context is correctly set.

Test 5:



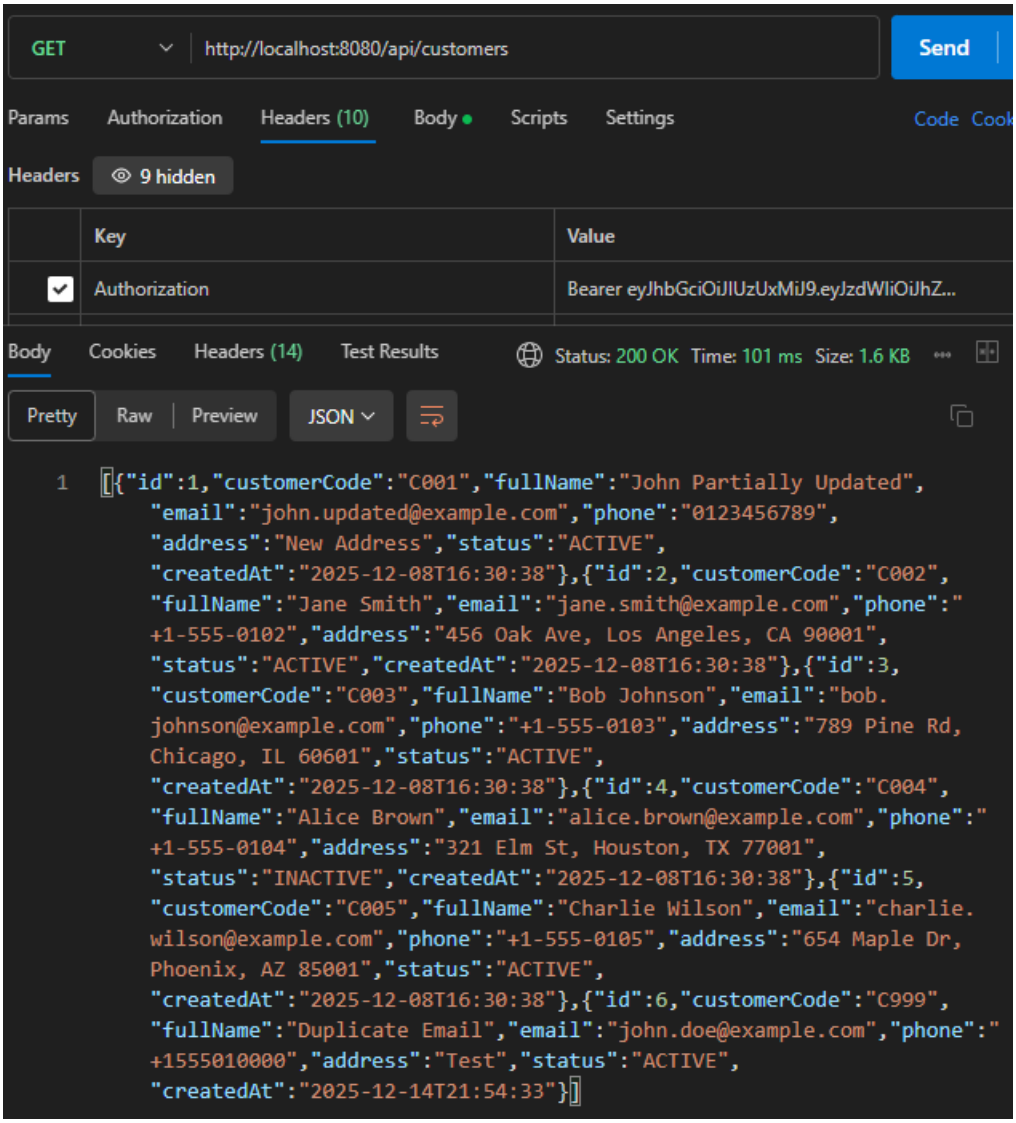
In a JWT-based stateless authentication system, logout is handled on the client side by removing the stored token.
The logout endpoint returns a simple message instructing the client to delete the token.
This confirms that the application follows the stateless JWT pattern instead of maintaining server-side sessions

Test 6:



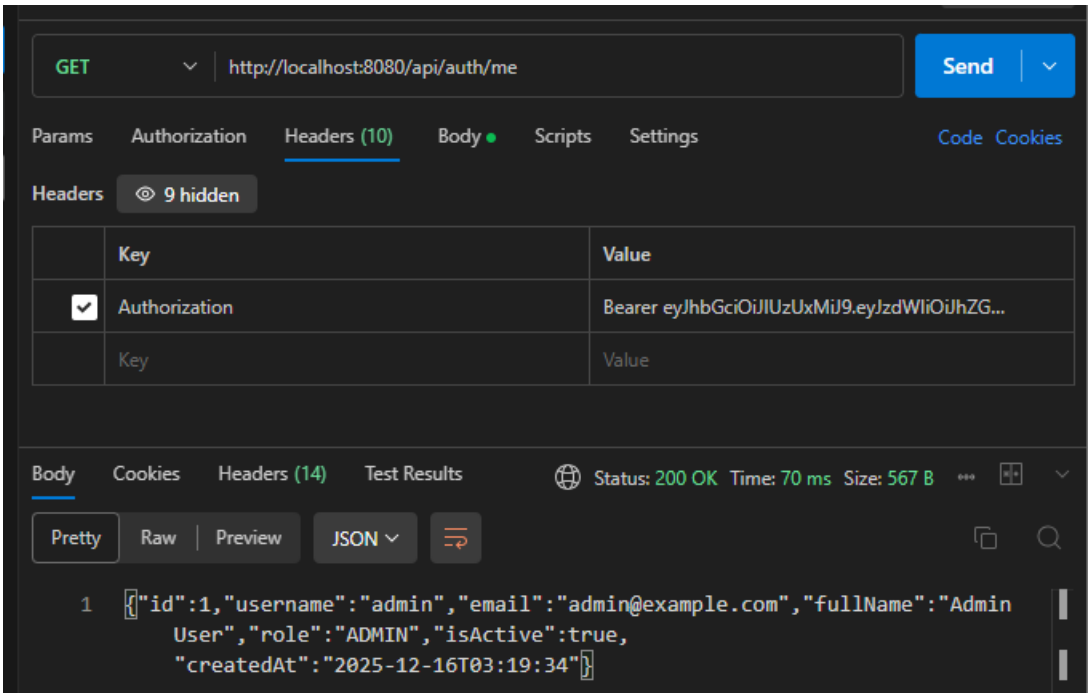
Since the security configuration requires authentication for GET /api/customers, any request without a valid JWT token is rejected.
The JwtAuthenticationEntryPoint returns an HTTP 401 response with a JSON error body indicating that authentication is required.
This confirms that unauthenticated users cannot access protected customer resources.

Test 7:



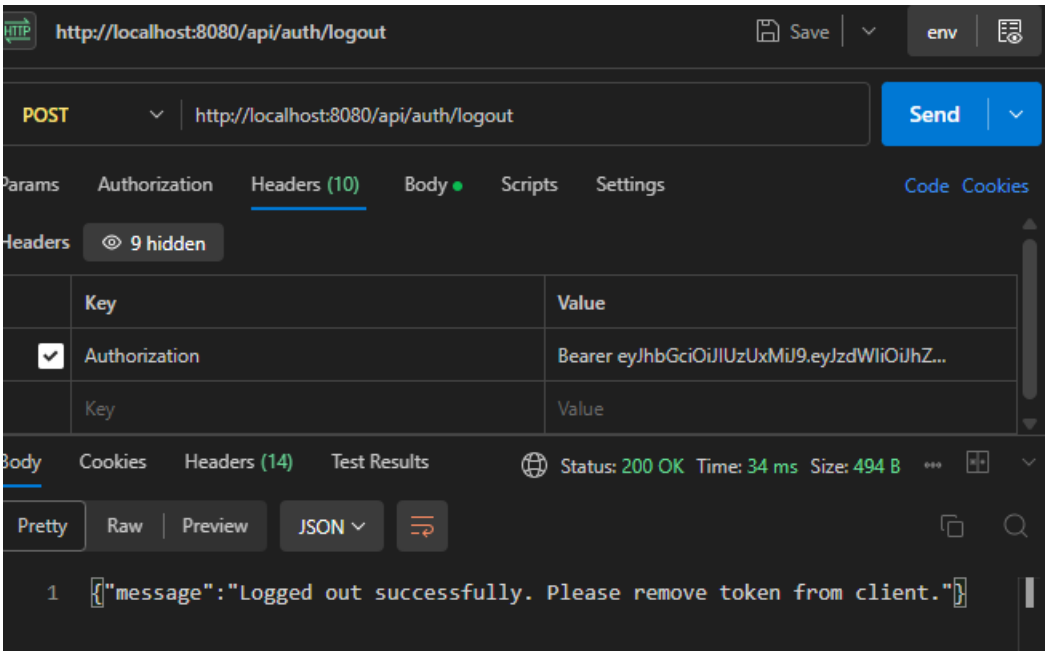
When a valid JWT is provided, the JwtAuthenticationFilter extracts and validates the token, loads the user details, and sets the authentication in the SecurityContext.
The request is then allowed to proceed, and the endpoint returns the list of customers with HTTP 200 OK.
This proves that authenticated users can successfully access protected endpoints.

Test 8:



first logged in to obtain a JWT and then called /api/auth/me with the header Authorization: Bearer <token>. The server returned 200 OK with the correct user information, which proves that the JWT is validated and the current user is correctly loaded from the SecurityContext. When I called the same endpoint without a token, it returned 401 Unauthorized, showing that the endpoint is properly protected

Test 9:

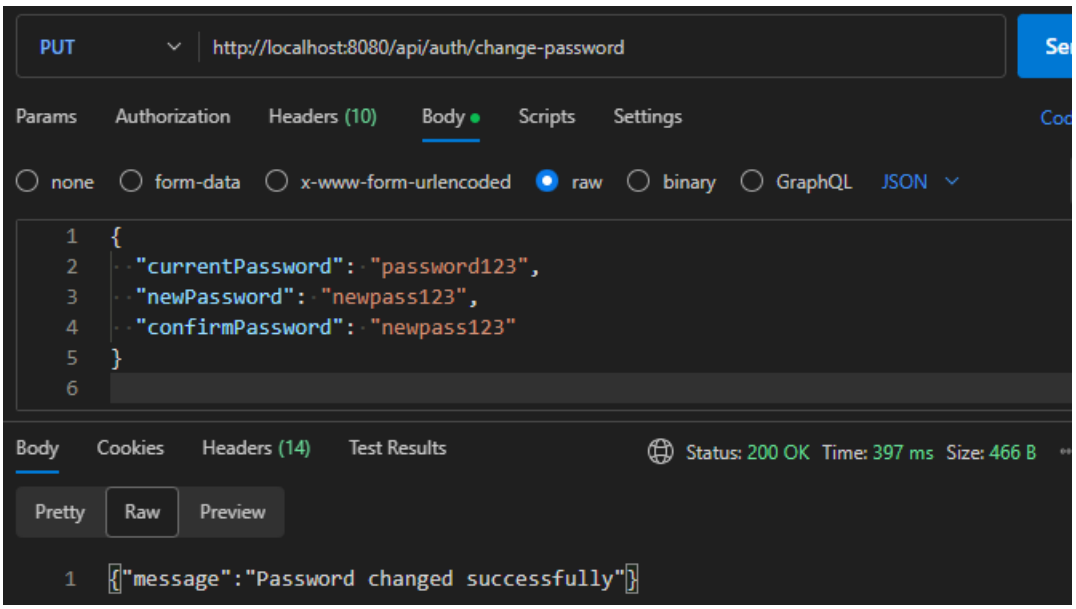


I sent a POST /api/auth/logout request (optionally with the JWT in the Authorization header). The API returned 200 OK and the message “Logged out successfully. Please remove token from client.” This confirms that the logout endpoint works as designed. Since JWT authentication is stateless, the token remains valid until it expires, so the client must delete the token locally to complete the logout process.

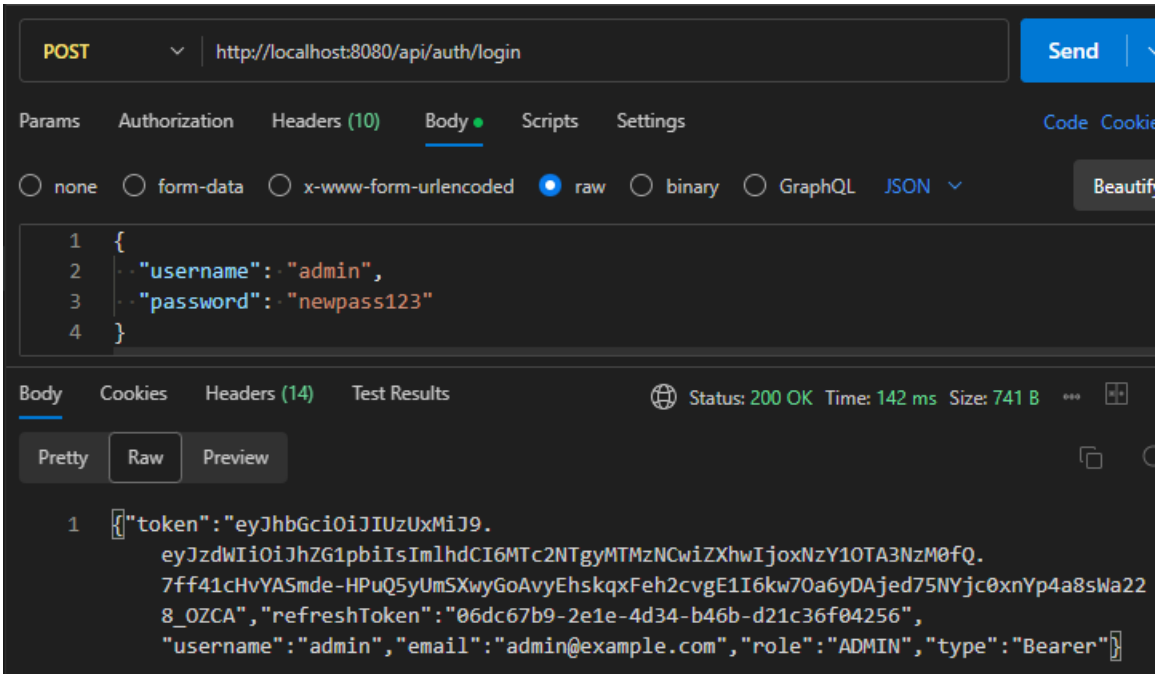
PartB:

Exercise 6: PASSWORD MANAGEMENT

6.1. Change Password

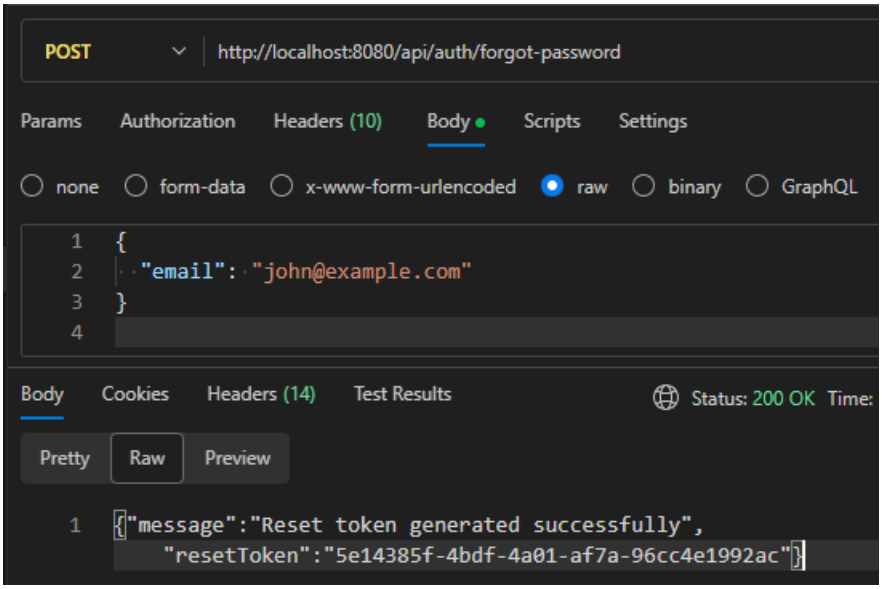


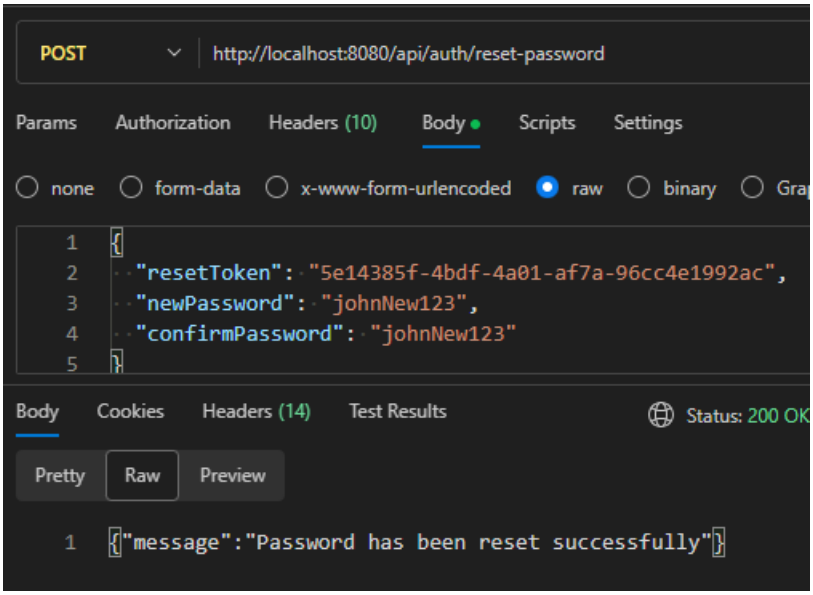
PUT /api/auth/change-password
This endpoint allows an authenticated user to update their password by sending the current password, a new password, and a confirmation of the new password in the request body. The server verifies the current password and that the two new passwords match, then securely hashes and saves the new password. On success, it returns HTTP 200 with the message "Password changed successfully"



POST /api/auth/login
This endpoint authenticates a user using a username and password. If the credentials are valid, the server returns HTTP 200 along with a JSON payload containing a JWT access token and a refresh token, plus basic user info (username, email, role, token type). These tokens are then used by the client to access protected resources and refresh the session without logging in again.

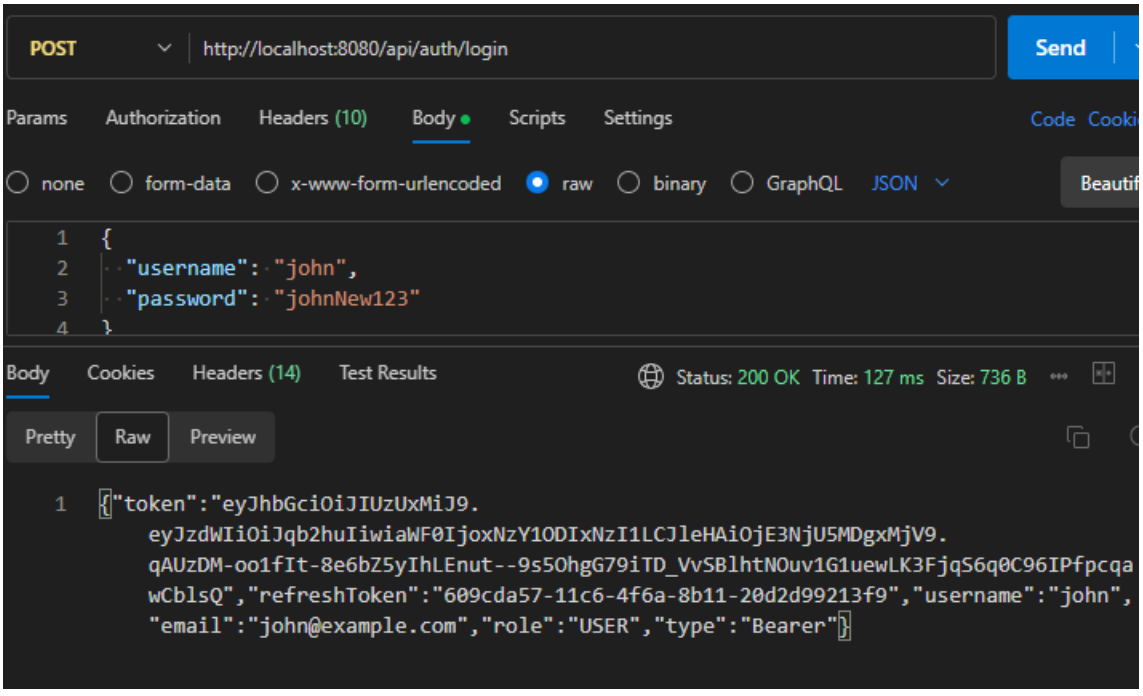
6.2. Forgot Password





Forgot / Reset Password flow

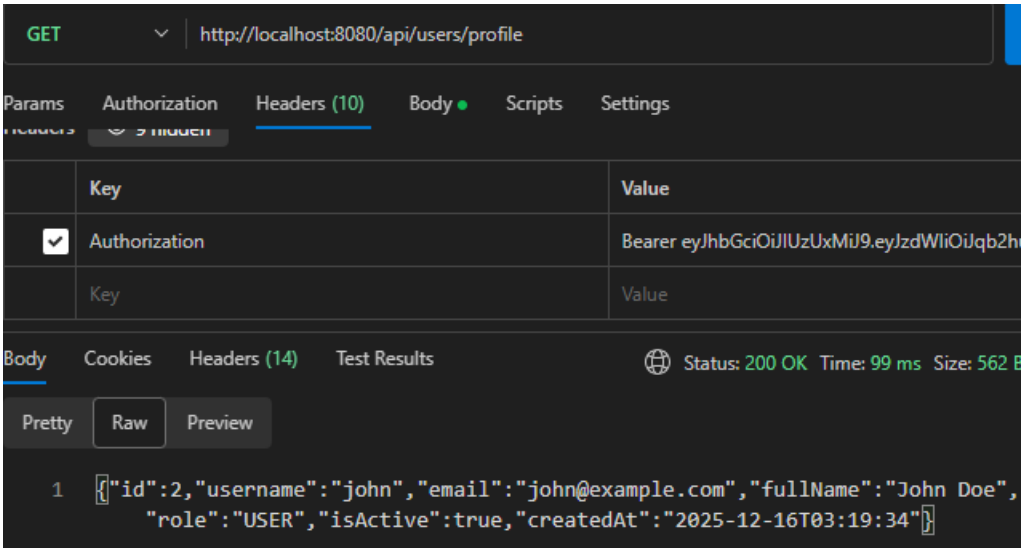
The POST /api/auth/forgot-password endpoint lets a user start the password-recovery flow by submitting their registered email; the server generates a one-time reset token with an expiry time and returns it in the response. The POST /api/auth/reset-password endpoint then uses that reset token plus the newPassword/confirmPassword fields to validate the token, ensure it hasn't expired, check the two passwords match, and safely update the user's stored (hashed) password.



After a successful reset, the user can call POST /api/auth/login again with their username and the new password. The server authenticates the updated credentials and returns a fresh JWT access token and refresh token, proving that the reset password is now active.

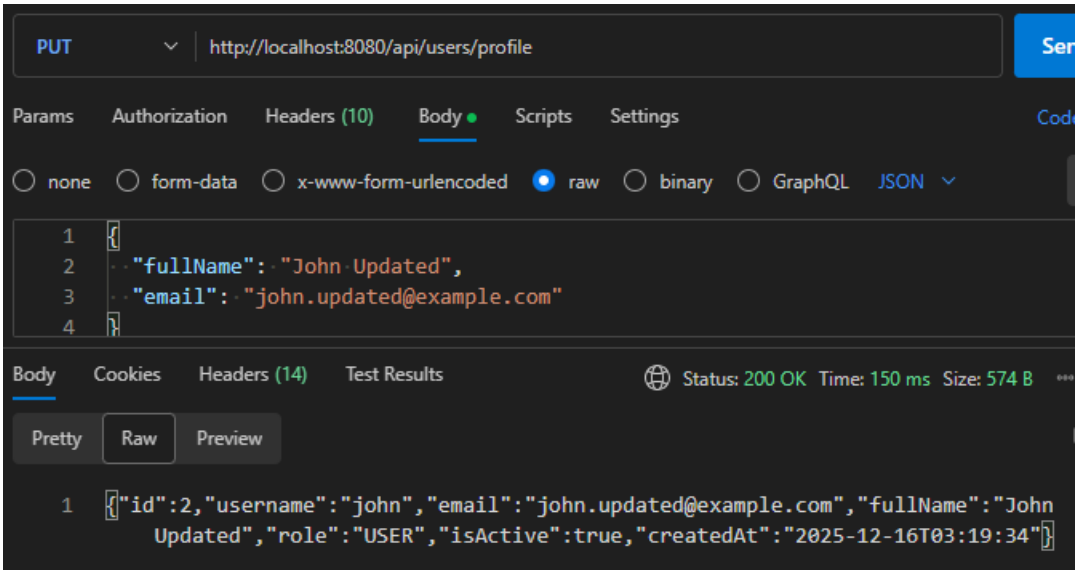
Exercise 7: USER PROFILE MANAGEMENT

7.1



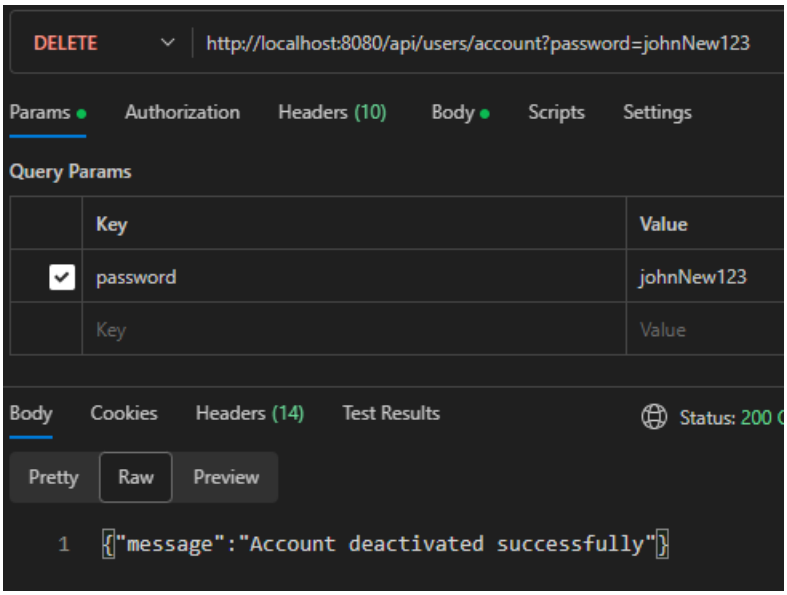
The GET /api/users/profile endpoint returns the authenticated user's profile based on the JWT in the Authorization header. When I call it with a valid token, the API responds with 200 OK and the user's basic information (username, email, fullName, role, isActive, createdAt).

7.2



The PUT /api/users/profile endpoint allows the logged-in user to update their full name and email. With a valid JWT and a valid body, the server returns 200 OK and the updated UserResponseDTO, confirming that the profile data was stored in the database

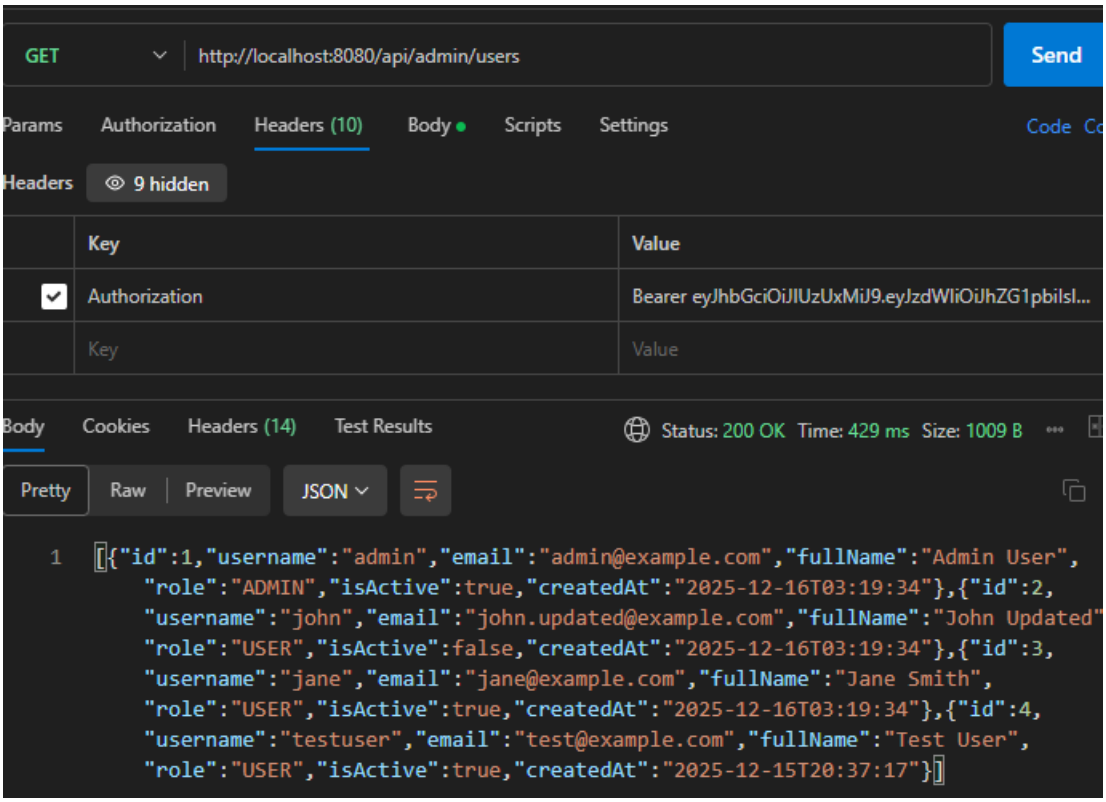
7.3



The DELETE /api/users/account endpoint performs a soft delete: it verifies the user's current password, and if it is correct, sets isActive = false for the current user. After a successful call, the user receives 200 OK with a success message, and subsequent login attempts for that account are rejected

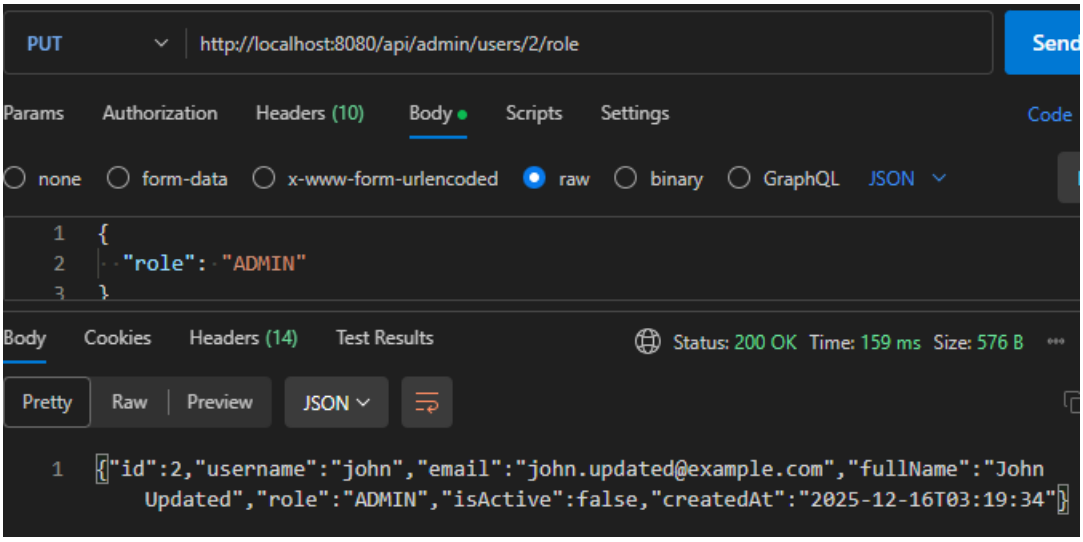
Exercise 8: ADMIN ENDPOINTS

8.1



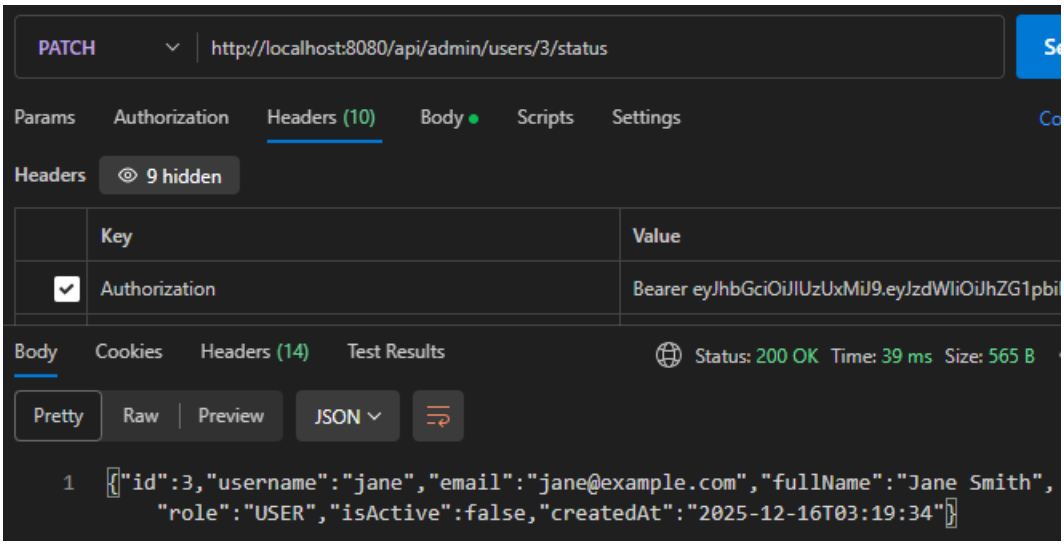
GET /api/admin/users is restricted to ADMIN. With a valid admin JWT token, the API returns 200 OK and a list of all users in the system as UserResponseDTO objects.

8.2



PUT /api/admin/users/{id}/role allows an admin to change a user's role. With a valid admin token and a body containing the new role, the API returns 200 OK and the updated user, showing the role field has changed.

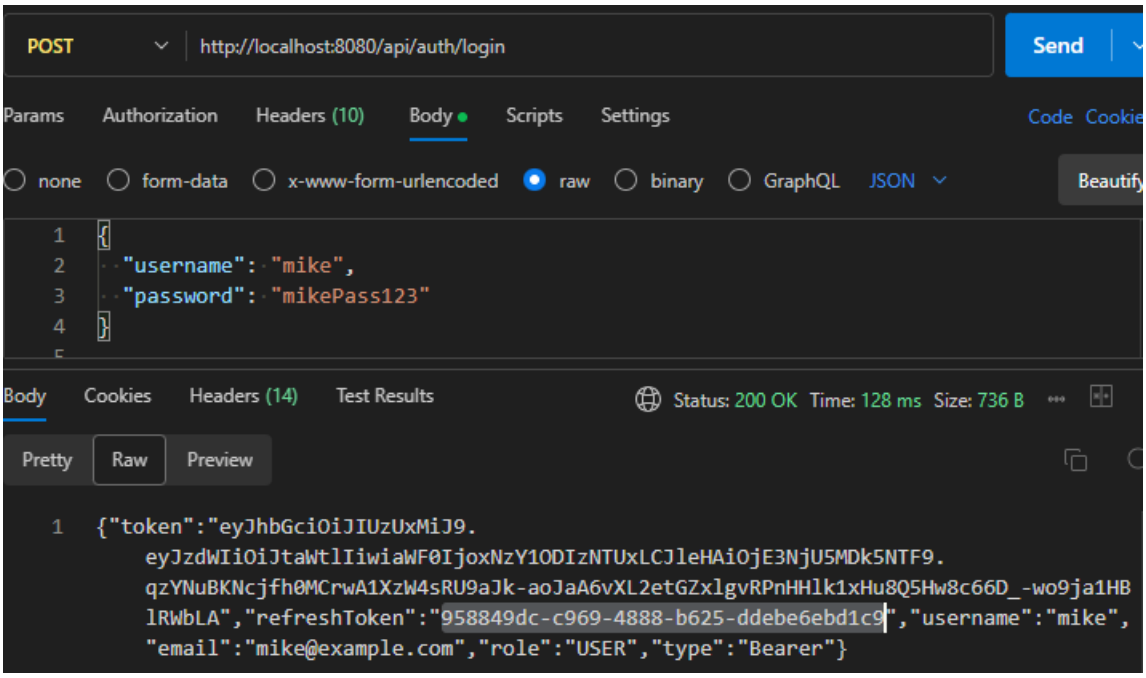
8.3



PATCH /api/admin/users/{id}/status toggles the isActive flag for a user. When called by an admin, the endpoint returns 200 OK and a UserResponseDTO where the isActive field is flipped, effectively deactivating or reactivating the account.

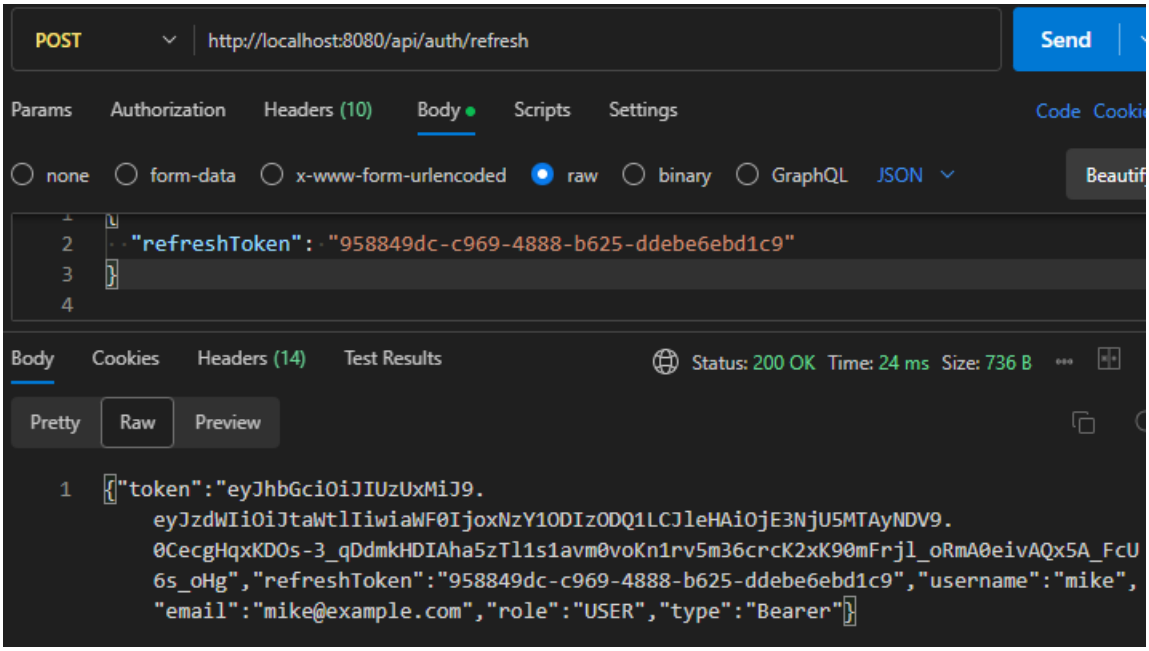
Exercise 9: REFRESH TOKEN

9.1



The client sends a valid username and password to POST /api/auth/login. The server authenticates the user, generates a short-lived access token (JWT) and a long-lived refresh token, stores the refresh token in the refresh_tokens table, and returns both tokens in the response. These tokens will be used for subsequent authenticated requests and for refreshing the access token when it expires.

9.2



This test calls POST /api/auth/refresh with a valid refresh token in the request body. The backend verifies that the refresh token exists in the database, is linked to an active user, and has not expired.

If the token is valid, the server generates a new access token (and optionally returns the same or a new refresh token) and sends it back to the client. The new access token can then be used in the Authorization: Bearer ... header to access protected endpoints.